

StutterCut: Uncertainty-Guided Normalised Cut for Dysfluency Segmentation

Suhita Ghosh¹, Melanie Jouaiti², Jan-Ole Perschewski¹, Sebastian Stober¹

¹Artificial Intelligence Lab, Otto-von-Guericke-University, Germany

²School of Computer Science, University of Birmingham, England

{suhita.ghosh, jan-ole.perschewski, stober}@ovgu.de, m.jouaiti@bham.ac.uk

Abstract

Detecting and segmenting dysfluencies is crucial for effective speech therapy and real-time feedback. However, most methods only classify dysfluencies at the utterance level. We introduce StutterCut, a semi-supervised framework that formulates dysfluency segmentation as a graph partitioning problem, where speech embeddings from overlapping windows are represented as graph nodes. We refine the connections between nodes using a pseudo-oracle classifier trained on weak (utterance-level) labels, with its influence controlled by an uncertainty measure from Monte Carlo dropout. Additionally, we extend the weakly labelled FluencyBank dataset by incorporating frame-level dysfluency boundaries for four dysfluency types. This provides a more realistic benchmark compared to synthetic datasets. Experiments on real and synthetic datasets show that StutterCut outperforms existing methods, achieving higher F1 scores and more precise stuttering onset detection.

Index Terms: stuttering, dysfluency, segmentation, semi-supervised

1. Introduction

Speech dysfluencies disrupt the natural flow of speech and diminish communicative ability and quality of life for those who stutter [1, 2]. Although deep learning has shown promise in detecting dysfluencies at the utterance level [3, 4], pinpointing the exact onset and boundaries remains understudied. Such precise localisation is crucial for targeted therapy [5], real-time feedback in clinical assessment [6] or self-guided contexts [7, 8], and generating realistic synthetic data [9].

A key obstacle is the scarcity of real-world datasets with strong (frame-level) dysfluency labels. Existing strong-labelled synthetic corpora, such as LibriStutter [10] and VCTK-based dysfluent corpora [9], often lack the natural variability of real speech and introduce unnatural artefacts. These include synthetic noise in prolongations [4], robotic distortions [9], and overly rigid pauses in blocks. In contrast, real stuttering exhibits subtle variations, such as fluctuating energy in repetitions and gradual transitions, which synthetic datasets fail to capture accurately.

Few methods address dysfluency segmentation. The most recent, YOLO-Stutter [9], applies the well-known region-based segmentation framework YOLO [11] to derive dysfluency boundaries. It leverages a pre-trained VITS [12] model to align speech and text embeddings, followed by spatial and temporal aggregation. However, its reliance on reference texts, which are often unavailable or inaccurate for atypical speech when using general ASR models such as Whisper [13], limits its real-world applicability. Another related approach by Harvill et al. [14] uses a frame-wise dysfluency detection model for stutter correction, utilising bidirectional LSTMs and fully connected layers. The method was evaluated for stutter correction solely by non-experts on Amazon Mechanical Turk, using just 20 samples per dysfluency

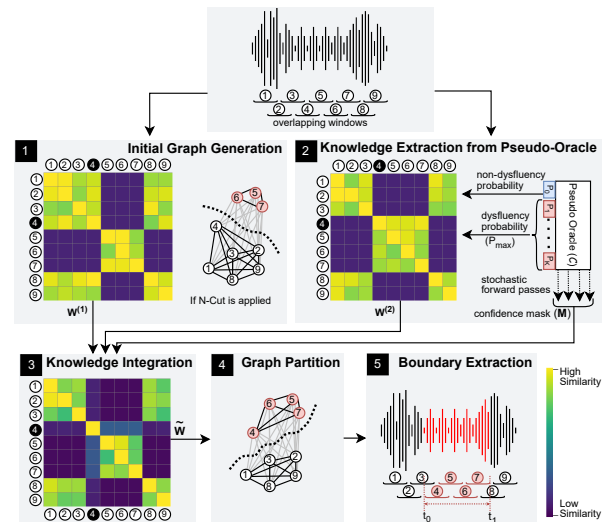


Figure 1: The input utterance is divided into overlapping windows, with embeddings from each window forming nodes in a graph. The connection strength of nodes is represented by colour intensity, with darker lines indicating stronger links. Dysfluent nodes and frames are highlighted in red. Notably, node 4, positioned at the boundary between fluent and dysfluent regions in the latent space, is initially misclassified as fluent but is correctly classified after integrating uncertainty-weighted classifier knowledge.

class and lacking rigorous objective accuracy assessment, making it challenging to determine its broader generalisability.

Thus, we introduce “StutterCut”, a graph-based soft-constrained clustering method [15], designed to segment dysfluent regions *without relying on strong dysfluency labels*. In our approach, embeddings from overlapping windows of an utterance serve as graph nodes, and edges represent the similarity between these embeddings. We use normalised cut (N-Cut) to partition the graph, whose dual consideration prevents over-fragmentation and ensures balanced segmentation [16]. To further refine node connectivity, we integrate knowledge from a pseudo-oracle classifier trained on weak dysfluency labels. This classifier mimics expert guidance by imposing soft segmentation constraints, with its influence adjusted by predictive uncertainty via Monte Carlo dropout entropy [17]. This ensures that only reliable predictions guide segmentation.

Additionally, we extend the weakly labelled FluencyBank [18] corpus by introducing FluencyBank++, which includes strong labels for four dysfluency types: prolongation, repetition, interjection, and block, offering a more authentic benchmark than synthetic datasets. Experiments on both synthetic and real-world stuttering datasets demonstrate that StutterCut effectively segments dysfluent regions,

outperforming existing methods in dysfluency segmentation. The code and FluencyBank++ dataset will be released online¹.

2. Normalised Cut (N-Cut)

Consider an undirected, weighted and fully-connected graph $G=(\mathcal{N},\mathcal{E})$, where $\mathcal{N}$ is the set of N nodes and $\mathcal{E}$ is the set of edges. Let $\mathbf{W}$ be the similarity matrix, where $\mathbf{W}_{i,j}$ is the weight of the edge between the nodes n_i and n_j , and let $\mathbf{D}$ be a diagonal matrix with entries $\mathbf{D}_i = \sum_j \mathbf{W}_{i,j}$. N-Cut [19] splits $\mathcal{N}$ into two disjoint subsets, $\mathcal{S}_1$ and $\mathcal{S}_2$. The membership of the nodes is given by $\mathbf{y} \in \{1, -b\}^N$, where b is a constant that ensures a balanced partition, preventing one subset from dominating the other [19]. N-Cut aims to partition the graph to minimize edges between subsets while preserving strong connectivity within each subset. It can be formulated as a generalised eigenvalue problem, whose *eigenvectors define the partition* [19]. Specifically, one solves: $\mathbf{D}^{-\frac{1}{2}}(\mathbf{D}-\mathbf{W})\mathbf{D}^{-\frac{1}{2}}=\lambda\mathbf{z}$, where $\mathbf{D}-\mathbf{W}$ is the graph Laplacian, known to be positive semidefinite, and $\mathbf{z}=\mathbf{D}^{-\frac{1}{2}}\mathbf{y}$, ensuring efficient eigen-decomposition [19]. The smallest eigenvector, $\mathbf{z}_0=\mathbf{D}^{-\frac{1}{2}}\mathbf{1}$ (corresponding to $\lambda=0$), is trivial and not useful for partitioning. Instead, the second smallest eigenvector, $\mathbf{z}_1$, also known as the **Fiedler vector**, provides the N-Cut solution (after substituting $\mathbf{z}=\mathbf{D}^{-\frac{1}{2}}\mathbf{y}$).

$$\mathbf{y}_1 = \underset{\mathbf{y}^T \mathbf{D} \mathbf{1} = 0}{\operatorname{argmin}_{\mathbf{y}}} \frac{\mathbf{y}^T (\mathbf{D}-\mathbf{W}) \mathbf{y}}{\mathbf{y}^T \mathbf{D} \mathbf{y}} \quad (1)$$

where the two $\mathcal{S}_1$ and $\mathcal{S}_2$ clusters are obtained by taking the sign or applying a threshold to $\mathbf{y}_1$ [19]. Finer clusters can also be obtained using higher-order eigenvectors [20].

3. StutterCut

We propose StutterCut, a method that segments dysfluent regions using only utterance-level (weak) dysfluency labels. Specifically, we refine the similarity matrix derived from Fiedler vectors by incorporating guidance from a pseudo-oracle classifier $\mathcal{C}$. The classifier is trained on weak labels, serving as a proxy for human expertise. This classifier introduces soft constraints, indicating which segments within an utterance are more likely to be grouped or separated. Unlike hard-constrained clustering [15], our soft constraints are incorporated by modulating the similarity matrix, enabling three key benefits: (1) independence from strong dysfluency labels, which are costly and time-consuming for experts to annotate, (2) flexibility to ignore misleading constraints, and (3) freedom to choose any clustering procedure. Our method comprises five stages, as shown in Figure 1:

1. Initial Graph Generation: We divide an utterance into N overlapping windows, each of length l with a stride of r . For each window, we extract embeddings using a pre-trained foundation model (such as WavLM [21]), whose representations capture the subtle acoustic and linguistic nuances essential for dysfluency detection, such as stretched sounds (in prolongations) and silent pauses caused by articulatory difficulty (in blocks). Each embedding corresponds to a node n_i in the graph G . We compute edge weights using the cosine similarity between node embeddings and construct the initial similarity matrix $\mathbf{W}^{(1)}$ from these weights. Intuitively, embeddings from the same class (dysfluent or non-dysfluent) tend to be closer in latent space [22], resulting in higher similarity values.

2. Knowledge Extraction from Pseudo-Oracle: At this stage, we derive another similarity matrix $\mathbf{W}^{(2)}$ from the predictions of the pseudo-oracle classifier $\mathcal{C}$, trained in a multilabel fashion using weak dysfluency labels. To avoid any data leakage and ensure real-world

applicability, $\mathcal{C}$ is trained on a separate dataset (or disjoint partition) from that used in the first stage, where only a pre-trained foundation model is used to extract embeddings. The similarity matrix $\mathbf{W}^{(2)}$ is derived from the cosine similarity of prediction probabilities, as shown in Eqn. (2). Here, $\mathbf{P}_0 \in \mathbb{R}^N$ is the probability vector for the non-dysfluent (fluent) class, where $\mathbf{P}_{i,0}$ denotes the probability of node n_i being fluent. Similarly, $\mathbf{P}_{\max} \in \mathbb{R}^N$ is defined by $\mathbf{P}_{i,\max} = \max_{k=1}^K \mathbf{P}_{i,k}$, capturing the highest predicted probability among the K dysfluent classes for node n_i . Since a window may contain both fluent and dysfluent segments, using both probabilities ensures a more informed distinction.

$$\mathbf{W}^{(2)} = \mathbf{P}_0 \mathbf{P}_0^T + \mathbf{P}_{\max} \mathbf{P}_{\max}^T \quad (2)$$

3. Prior Knowledge Integration from Pseudo-Oracle: At this stage, we derive the refined similarity matrix $\widetilde{\mathbf{W}}$ by fusing $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(2)}$ according to:

$$\widetilde{\mathbf{W}} = \mathbf{W}^{(1)} \odot (\mathbf{I} - \mathbf{D}_M) + (\mathbf{W}^{(1)} \mathbf{W}^{(2)}) \odot \mathbf{D}_M \quad (3)$$

where $\mathbf{I} \in \mathbb{R}^{N \times N}$ is an identity matrix, and $\mathbf{D}_M = \operatorname{diag}(\mathbf{M})$ is a diagonal matrix formed from the N -dimensional mask $\mathbf{M}$, which controls the contribution of the pseudo-oracle. The mask value $\mathbf{M}_i$ represents the confidence of the pseudo-oracle’s prediction for node n_i and is derived as $\mathbf{M}_i = 1 - \frac{\mathbf{H}_i}{\mathbf{H}_{\max}}$, where $\mathbf{H}_i$ is the entropy estimated from multiple stochastic forward passes (MC dropout) [17] and, $\mathbf{H}_{\max} = \log K$, denotes the maximum possible entropy for K classes. The scaling by $\mathbf{H}_{\max}$ ensures that $\mathbf{M}_i$ remains within the range $[0,1]$. Finally, to strengthen highly similar connections while maintaining overall graph connectivity, we threshold by setting similarities below τ to a small value (10^{-5}), ensuring that weaker connections are preserved but providing minimal influence.

4. Graph Partition: We apply N-Cut to $\widetilde{\mathbf{W}}$ to produce two clusters: dysfluent and non-dysfluent. The partitioning is determined using the signs of the Fiedler vector $\mathbf{y}_1$ or its average value $\bar{\mathbf{y}}_1$, following the approach in [16]. Formally, $\mathcal{S}_1 = \{n_i \mid \mathbf{y}_{1,i} \leq \phi\}$, $\mathcal{S}_2 = \{n_i \mid \mathbf{y}_{1,i} > \phi\}$, where $\phi \in \{0, \bar{\mathbf{y}}_1\}$. Unlike in image-based methods [16], where the cluster with the highest Fiedler vector identifies the salient object due to a clear foreground-background distinction, speech lacks such separation. Instead, we determine the dysfluent cluster as the one with a higher number of dysfluent nodes, identified using $\mathcal{C}$.

5. Boundary Extraction (BE): We derive the boundaries of dysfluencies by merging consecutive dysfluent windows into continuous segments, taking the earliest start time t_0 and the latest end time t_1 , as shown in Figure 1. Short gaps ($\leq \eta$ ms) are merged to reduce fragmentation, and isolated short segments ($\leq \eta$ ms) are removed to limit false positives.

4. FluencyBank++ Dataset

One of the major challenges in dysfluency segmentation is the scarcity of real-world datasets. To address this limitation, we extend FluencyBank [18] with strong dysfluency labels. We call our augmented version FluencyBank++.

Annotation Protocol: FluencyBank consists of 4,144 three-second clips from 33 podcasts (23 male, 10 female speakers) annotated for five dysfluency types: block, interjection, prolongation, sound repetition, and word repetition. We combine sound, word, syllable, and phrase repetitions into a single category: *repetitions*. Further, we filter out clips lacking speech or containing music and retain only those annotated as dysfluent by at least two annotators.

Annotating dysfluency boundaries is challenging because transitions between dysfluencies are often fuzzy. For example, interjections such as “um” can be repeated and may blend with

¹github.com/suhitaghosh10/stuttercut

sound repetitions or prolongations, and multiple dysfluency types frequently occur together [23]. To improve annotation quality, we employ a multi-stage strategy with five experts: two dysfluency specialists for the initial stages and three speech pathologists for final refinement. The three stages are:

- 1. Utterance-Level Correction:** Original three-second clips are extended to five seconds (adding one second on each side) to avoid truncating dysfluencies, especially partial word repetitions, reported in [4]. Two annotators review these extended clips and revise the weak labels, adjusting for any shifts introduced due to the extension.
- 2. Frame-Level Boundary Annotation:** Both annotators independently mark start and end points for each dysfluency in the clips approved in the previous step. For utterances with multiple dysfluencies (e.g., block and interjection), each type is presented separately, and the annotator is explicitly informed which dysfluency type to mark. This approach ensures unbiased and structured boundary annotation.
- 3. Boundary Refinement and Consensus:** Three speech pathologists review the dysfluency boundaries from the previous stage, ensuring no regions are missed. A voting system was implemented, where the frames marked as dysfluent by at least two annotators were finally labelled as dysfluent.

Dataset Statistics: FluencyBank++ consists of 3,017 five-second clips, which is 72.8% of the original data after filtering. Table 1 provides an overview of each dysfluency class, including the number of clips and duration ranges. To assess annotation reliability, we calculated Fleiss’ Kappa over 0.1 s frames following the procedure in [20]. The Fleiss’ Kappa score improved from 0.47 (moderate agreement) in the second stage to 0.71 (strong agreement) in the final stage, demonstrating that the multi-level process enhanced annotation reliability.

Table 1: Overview of the FluencyBank++ dataset.

Dysfluency	#Clips	Minimum Duration (s)	Maximum Duration (s)
Interjection	1,130	0.12	1.88
Repetition	921	0.20	4.99
Block	530	0.23	4.20
Prolongation	436	0.41	3.95

5. Experiment and Results

Datasets: All speech recordings are resampled to 16 kHz. For our experiments, we consider FluencyBank++, our newly introduced dataset, along with two English stuttering datasets: VCTK-TTS [9] and Sep-28K [24]. VCTK-TTS is a synthetic, strong-labelled dataset extending VCTK [25], comprising utterances from 109 native English speakers with various accents and artificially inserted dysfluencies such as repetitions, blocks, missing words, and prolongations. Sep-28K is a large-scale dysfluency dataset having the same weak dysfluency labels as in FluencyBank. Since Sep-28K and FluencyBank++ do not include ‘missing words’, we exclude this dysfluency class from our analysis.

Baselines: We compare StutterCut against four baselines: YOLO-Stutter [9], the current state-of-the-art in dysfluency segmentation; Harvill et al. [14], a stutter-correction model closely related to dysfluency segmentation; and two variants of Whister [4], a classifier that detects the type of dysfluency by leveraging Whisper representations as input features. One variant of Whister, Whister_{ML}, is trained using focal loss in a multi-label classification setup with sigmoid activation instead of softmax, while the other, Whister_{MC}, follows the multi-class classification approach with weighted cross-entropy loss as described in [4]. Since Whister-based models are trained on weak dysfluency labels, they cannot provide explicit boundary information. Therefore, we extract boundaries

by applying the boundary extraction approach (see Section 3) after predicting dysfluency over overlapping windows. Notably, Whister_{ML} also serves as the pseudo-oracle $\mathcal{C}$ in our method.

Data Selection and Split: The Whister-based models and the method by Harvill et al. [14] are trained on weak labels. We train these models on VCTK-TTS and Sep-28K, excluding LibriStutter [10] since its unnatural dysfluencies have been shown to degrade performance in our initial experiments and prior work [4]. We use the same data split for training and validation for all models, with no speaker overlap, ensuring fair evaluation. For Sep-28K, we follow the splits from [4], using 1,400 clips from two podcasts (“I Stutter So What” and “My Stuttering Life”) for validation and the remainder for training. However, Sep-28K is excluded from the dysfluency segmentation evaluation due to the absence of strong labels. For VCTK-TTS, we use 118,085 clips from 60 speakers for training, 12 speakers for validation and the rest for evaluation. We use the entire FluencyBank++ dataset for evaluation.

Training and Hyperparameters: The models trained on weak labels use the Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$) with a learning rate of 10^{-4} . They are trained with a batch size of 1024 for 100 epochs, halving the learning rate if the validation loss stagnates for 2 epochs, following [4]. Training each model takes ≈ 48 h on an Nvidia A100 (80GB) GPU. For YOLO-Stutter, we extract speech encodings using the pre-trained VITS [12] model, obtain transcriptions with WhisperX [26], and perform segmentation using a model pre-trained on VCTK-TTS, following the set-up in [9].

For StutterCut, we set a similarity threshold of $\tau = 0.25$. We employ overlapping windows of length $l = 0.75$ s and a stride of $r = 0.1$ s, empirically determined on the validation set. This configuration provides high temporal resolution for detecting brief interjections while preserving sufficient context for longer dysfluencies such as word repetitions. The 0.1 s stride creates overlapping windows, allowing the method to capture subtle transitions more effectively and enhance boundary detection. MC dropout-based uncertainty measures are computed from 100 stochastic forward passes.

Inference and Computational Efficiency: Using $l = 0.75$ s and $r = 0.1$ s, StutterCut generates dysfluency boundaries in about 2.2 s for a 1 min utterance. Our method uses Whister_{ML} as the pseudo-oracle, which has 7.1 million trainable parameters, lesser than YOLO-Stutter’s 33 million [9].

Evaluation Metrics: We evaluate performance using three metrics: time-F1 (t-F1), time-recall (t-recall), and onset error. T-F1 and t-recall [9] assess boundary alignment by computing the Intersection over Union (IoU) between predicted and ground-truth intervals, considering a detection true positive if $\text{IoU} > 0.5$. T-F1 balances pre-

Table 2: Performance of methods using different embeddings and clustering algorithms, with overall T-F1 and t-recall scores reported with 95% confidence intervals.

Embedding	Method	FluencyBank++		VCTK-TTS	
		T-F1 $\uparrow$	T-recall $\uparrow$	T-F1 $\uparrow$	T-recall $\uparrow$
Whisper	K-Means	59.9 $\pm$ 4.5	80.9 $\pm$ 4.6	78.5 $\pm$ 3.8	80.8 $\pm$ 4.3
	FuzzyCMeans	60.7 $\pm$ 4.6	81.2 $\pm$ 4.6	78.5 $\pm$ 3.9	80.6 $\pm$ 4.4
	Fiedler [$\geq \bar{y}_1$]	63.0 $\pm$ 4.1	82.9 $\pm$ 4.2	79.4 $\pm$ 3.4	100.0 $\pm$ 0.0
	Fiedler [$y_1 \geq 0$]	63.5 $\pm$ 3.9	83.2 $\pm$ 4.6	82.7 $\pm$ 3.5	88.0 $\pm$ 3.7
WavLM	K-Means	59.7 $\pm$ 3.9	80.7 $\pm$ 4.1	75.1 $\pm$ 3.0	86.9 $\pm$ 4.0
	FuzzyCMeans	61.9 $\pm$ 4.5	81.3 $\pm$ 4.6	75.4 $\pm$ 2.7	87.0 $\pm$ 4.0
	Fiedler [$\geq \bar{y}_1$]	63.1 $\pm$ 4.8	82.6 $\pm$ 4.7	76.4 $\pm$ 1.0	100.0 $\pm$ 0.0
	Fiedler [$y_1 \geq 0$]	63.2 $\pm$ 4.7	82.8 $\pm$ 5.0	76.8 $\pm$ 3.4	88.4 $\pm$ 4.6
Wav2Vec2	K-Means	56.3 $\pm$ 4.5	79.4 $\pm$ 4.7	71.8 $\pm$ 3.9	82.0 $\pm$ 4.0
	FuzzyCMeans	59.4 $\pm$ 4.4	80.6 $\pm$ 4.3	72.0 $\pm$ 3.9	81.7 $\pm$ 4.1
	Fiedler [$\geq \bar{y}_1$]	59.5 $\pm$ 4.5	82.2 $\pm$ 4.7	72.3 $\pm$ 3.5	100.0 $\pm$ 0.0
	Fiedler [$y_1 \geq 0$]	60.2 $\pm$ 4.7	82.0 $\pm$ 4.9	72.9 $\pm$ 3.8	86.0 $\pm$ 3.9

Table 3: 95% confidence intervals are reported for all models. The first four methods are baselines, and the remaining entries are Stutter-Cut variants: SCut (using classifier guidance weighted by MC dropout entropy), SCut [$M_{p \geq 0.5}$] (using classifier guidance weighted by MC dropout probability), SCut [$-M$] (using classifier guidance without mask), and SCut [$-M-\mathcal{C}$] (pure N-Cut). “Prolong.” (prolongation), “Rep.” (repetition), “Interj.” (interjection), “Block”, “S.Rep.” (sound repetition), and “W.Rep.” (word repetition) denote dysfluency-class-wise t-F1 scores. T-F1, t-recall, and onset error (Onset E., measured in seconds) represent the overall scores.

Method	FluencyBank++							VCTK-TTS						
	Prolong. $\uparrow$	Rep. $\uparrow$	Interj. $\uparrow$	Block $\uparrow$	T-F1 $\uparrow$	T-recall $\uparrow$	Onset E. $\downarrow$	Prolong. $\uparrow$	S. Rep $\uparrow$	W. Rep $\uparrow$	Block $\uparrow$	T-F1 $\uparrow$	T-recall $\uparrow$	Onset E. $\downarrow$
YOLO-Stutter [9]	60.2 $\pm$ 4.5	59.5 $\pm$ 3.3	44.2 $\pm$ 5.1	63.4 $\pm$ 8.6	56.8 $\pm$ 3.4	87.1 $\pm$ 4.7	0.5 $\pm$ 0.6	53.6 $\pm$ 3.4	59.1 $\pm$ 5.4	65.6 $\pm$ 7.4	56.8 $\pm$ 4.5	58.8 $\pm$ 3.7	100.0 $\pm$ 0.0	1.4 $\pm$ 0.7
Harvill et al. [14]	10.2 $\pm$ 10.9	39.5 $\pm$ 16.7	23.2 $\pm$ 14.9	13.4 $\pm$ 11.6	21.6 $\pm$ 14.8	47.1 $\pm$ 13.7	1.5 $\pm$ 0.6	21.4 $\pm$ 23.7	29.2 $\pm$ 12.3	25.8 $\pm$ 10.8	24.9 $\pm$ 13.7	25.3 $\pm$ 13.4	67.0 $\pm$ 12.3	1.7 $\pm$ 0.7
Whistervsc [4]	43.5 $\pm$ 6.3	35.4 $\pm$ 7.3	54.1 $\pm$ 6.5	41.9 $\pm$ 4.6	43.7 $\pm$ 5.7	55.5 $\pm$ 6.7	1.4 $\pm$ 0.5	27.4 $\pm$ 8.7	37.2 $\pm$ 11.3	34.8 $\pm$ 10.8	47.9 $\pm$ 13.4	36.8 $\pm$ 7.4	77.0 $\pm$ 9.1	1.3 $\pm$ 1.1
Whistervml	67.5 $\pm$ 4.3	65.8 $\pm$ 4.4	57.3 $\pm$ 4.5	53.8 $\pm$ 4.7	61.1 $\pm$ 4.8	88.5 $\pm$ 4.3	0.4 $\pm$ 0.5	63.1 $\pm$ 4.2	77.6 $\pm$ 4.1	65.0 $\pm$ 4.9	76.5 $\pm$ 4.2	70.6 $\pm$ 4.7	86.7 $\pm$ 4.1	0.4 $\pm$ 0.3
SCut	73.2 $\pm$ 4.0	68.8 $\pm$ 4.3	64.5 $\pm$ 4.5	70.6 $\pm$ 4.4	69.3 $\pm$ 4.9	89.2 $\pm$ 4.7	0.3 $\pm$ 0.5	84.7 $\pm$ 3.5	91.5 $\pm$ 2.2	87.6 $\pm$ 3.1	87.6 $\pm$ 3.3	87.2 $\pm$ 3.1	92.6 $\pm$ 3.2	0.2 $\pm$ 0.4
SCut [$M_{p \geq 0.5}$]	68.7 $\pm$ 4.3	65.5 $\pm$ 4.4	57.2 $\pm$ 5.7	63.3 $\pm$ 4.3	63.7 $\pm$ 4.4	86.6 $\pm$ 4.5	0.4 $\pm$ 0.5	84.4 $\pm$ 3.0	90.2 $\pm$ 2.1	84.7 $\pm$ 3.7	87.3 $\pm$ 3.0	86.6 $\pm$ 3.3	91.6 $\pm$ 3.3	0.2 $\pm$ 0.4
SCut [$-M$]	68.6 $\pm$ 4.2	63.0 $\pm$ 4.2	57.4 $\pm$ 4.4	64.7 $\pm$ 4.0	63.4 $\pm$ 1.1	83.2 $\pm$ 2.2	0.3 $\pm$ 0.4	83.5 $\pm$ 3.5	83.0 $\pm$ 3.3	81.2 $\pm$ 3.1	83.0 $\pm$ 3.7	82.7 $\pm$ 3.5	88.0 $\pm$ 3.7	0.5 $\pm$ 0.8
SCut [$-M-\mathcal{C}$]	68.2 $\pm$ 4.3	64.8 $\pm$ 4.5	54.5 $\pm$ 4.7	63.6 $\pm$ 4.4	62.8 $\pm$ 4.9	85.2 $\pm$ 4.8	0.4 $\pm$ 0.5	83.0 $\pm$ 5.5	89.5 $\pm$ 2.5	87.3 $\pm$ 5.3	85.7 $\pm$ 3.2	86.1 $\pm$ 3.1	90.1 $\pm$ 3.2	0.3 $\pm$ 0.4

cision while t-recall measures the proportion of correctly identified dysfluency segments. Onset error, defined as the absolute difference between predicted and actual onset times quantifies the accuracy of dysfluency onset detection.

Dysfluency-class-wise scores are computed by first calculating the metric for each speaker in the test set and then macro-averaging these scores across speakers, which prevents bias from speakers with more data and ensures equal representation [4]. Overall performance is obtained by macro-averaging these dysfluency-class-wise scores to mitigate class imbalance, particularly in FluencyBank++, where interjections occur over twice as often as prolongations. This approach treats all dysfluency classes equally and aligns with prior work on unbalanced datasets [27, 28].

Selection of Embeddings and Clustering Method: We explore different combinations of clustering methods and speech embeddings to determine the optimal setup for StutterCut. Specifically, we compare the graph-based N-Cut method with traditional clustering approaches (K-means and Fuzzy C-means), leveraging embeddings from foundation models known for capturing dysfluency representations [4, 29, 30]. We consider the embeddings from the ASR-based foundation model Whisper alongside the self-supervised models WavLM Large [21] and Wav2Vec2 [31]. Following [4], we extract Whisper embeddings from all encoder layers, while for Wav2Vec2 and WavLM we use the 12th and 20th layers, respectively, as these better capture the nuances essential for dysfluency detection [29, 30]. Finally, we evaluate two Fiedler thresholding variants: one using a sign threshold (Fiedler [$y_1 \geq 0$]) and the other using the mean Fiedler vector as the threshold (Fiedler [$\geq \bar{y}_1$]).

Table 2 shows that Fiedler-based methods consistently outperform K-Means and Fuzzy C-Means in t-F1 and t-recall. Regardless of the type of embeddings, Fiedler [$\geq \bar{y}_1$] achieves the highest mean t-recall (100.0%) on VCTK-TTS but with lower precision due to synthetic noise misclassification. In contrast, Fiedler [$y_1 \geq 0$] reduces over-detection, improving precision at a slight cost to mean t-recall. As the variations in onset error were minimal (0.3 s to 0.8 s) for both datasets, detailed results are omitted from the table. Ultimately, we consider Whisper embeddings with Fiedler [$y_1 \geq 0$], as they provide the best mean t-F1 for both of the datasets.

Comparison to Baselines: Table 3 demonstrates the effectiveness of StutterCut. Our best variant, SCut, using classifier guidance weighted by MC dropout entropy, achieves the highest overall performance on both FluencyBank++ and VCTK-TTS. Among baselines, YOLO-Stutter shows high mean t-recall (87.1% in FluencyBank++, 100.0% in VCTK-TTS) but low overall mean t-F1 (56.8% in FluencyBank++, 58.8% in VCTK-TTS) due to over-detection of dysfluencies. The method by Harvill et al. [14] struggles significantly, especially in prolongation detection, making it the weakest. Whistervml, also used as the pseudo-oracle in our approach, outperforms other baselines but has difficulty with the block (mean t-F1: 53.8%) and interjection detection (mean t-F1:

57.3%) on FluencyBank++. SCut improves overall mean t-F1 over Whistervml by 13.4% on FluencyBank++ and 24.1% on VCTK-TTS. As both methods use Whisper embeddings, the improvement stems from the synergy of graph-based clustering and classifier-guided constraints. Nonetheless, StutterCut struggles with interjections (e.g., “um”, “like”, “aah”) that lack pauses or articulatory struggle, often failing to detect them. Additionally, blocks are often misclassified as pauses, an issue noted in prior dysfluency classification works [4].

Ablation Study: We explore an alternative uncertainty-weighting variant, SCut [$M_{p \geq 0.5}$], where confidence is based on the mean probability computed over stochastic MC passes [17]. However, it proves less effective than entropy-based weighting, as softmax probabilities tend to be overconfident [32].

Retaining classifier guidance but removing the entropy uncertainty mask (SCut [$-M$]) lowers the overall mean t-F1 of our best-performing variant SCut from 69.3% to 63.4% on FluencyBank++ and from 87.2% to 82.7% on VCTK-TTS. Disabling both classifier constraints and uncertainty weighting (SCut [$-M-\mathcal{C}$]), which corresponds to the pure N-Cut method, further degrades performance, reducing mean t-F1 by 10% and t-recall by 5% on FluencyBank++, reaffirming the necessity of uncertainty-weighted classifier guidance. Interestingly, in VCTK-TTS, the pure N-Cut method (SCut [$-M-\mathcal{C}$]) already surpasses Whistervml, with minimal gains from MC entropy-weighted $\mathcal{C}$ priors. This is likely due to the synthetic nature of VCTK-TTS dysfluencies, where predictable patterns and artificial cues simplify segmentation, leading to consistently higher t-F1 and t-recall compared to the FluencyBank++ dataset.

6. Conclusion and Future Work

We introduce StutterCut, a semi-supervised graph-based framework for dysfluency segmentation that operates without strong labels or transcriptions, making it more applicable to real-world scenarios. Our method strikes an optimal balance between recall and precision, outperforming conventional clustering and supervised deep learning baselines on real and synthetic datasets. We also present FluencyBank++, a dataset with frame-level labels for four dysfluency types that, although limited in size for training, provides a realistic benchmark compared to synthetic datasets.

Future work will explore alternatives to weakly supervised classifiers for cluster identification, such as higher-order statistics from the Fiedler vector. We also aim to develop adaptive window sizing, as our preliminary observations indicate that the same temporal resolution is not optimal for all dysfluency types (e.g., longer windows for word repetitions and shorter ones for interjections). Additionally, we plan to incorporate acoustic features to better distinguish interjections and blocks from fluent speech and to accurately separate overlapping dysfluencies. Finally, we aim to evaluate StutterCut on multilingual stuttering corpora to further establish its generalisability.

7. Acknowledgements

This research has been supported by the Federal Ministry of Education and Research of Germany through the projects Medinym (focused on AI-based anonymisation of personal patient data in clinical text and voice datasets) and Anti-Stotter (overcoming stuttering with AI).

8. References

- [1] J. Lian and G. Anumanchipalli, "Towards hierarchical spoken language dysfluency modeling," *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*, 2024.
- [2] A. Craig, E. Blumgart, and Y. Tran, "The impact of stuttering on the quality of life in adults who stutter," *Journal of Fluency Disorders*, vol. 34, no. 2, pp. 61–71, 2009.
- [3] S. P. Bayerl, M. Gerczuk, A. Batliner, C. Bergler, S. Amiriparian, B. Schuller, E. Nöth, and K. Riedhammer, "Classification of stuttering—the ComParE challenge and beyond," *Computer Speech & Language*, vol. 81, p. 101519, 2023.
- [4] V. Changawala and F. Rudzicz, "Whister: Using Whisper's representations for stuttering detection," in *Interspeech 2024*, vol. 2024, 2024, pp. 897–901.
- [5] J. Freeburn, "Speech therapy: being understood clearly," in *Functional Movement Disorder: An Interdisciplinary Case-based Approach*. Springer, 2022, pp. 341–352.
- [6] A. Laiho, H. Elovaara, K. Kaisamatti, K. Luhtalampi, L. Talaskivi, S. Pohja, K. Routamo-Jaattela, and E. Vuorio, "Stuttering interventions for children, adolescents, and adults: a systematic review as a part of clinical guidelines," *Journal of Communication Disorders*, vol. 99, p. 106242, 2022.
- [7] J. C. Saldanha and R. Pinto, "Real time feedback system for speech dysfluency in children," in *Smart Sensors Measurement and Instrumentation: Select Proceedings of CISCON*. Springer, 2023, pp. 75–91.
- [8] B. Belke, "Ego dystonic voice conversion for reducing stuttering," PCT Patent WO2024/200875A1, 2024. [Online]. Available: <https://patents.google.com/patent/WO2024200875A1/en>
- [9] X. Zhou, A. Kashyap, S. Li, A. Sharma, B. Morin, D. Baquirin, J. Vonk, Z. Ezze, Z. Miller, M. Tempini, J. Lian, and G. Anumanchipalli, "YOLO-Stutter: End-to-end region-wise speech dysfluency detection," in *Interspeech 2024*, 2024, pp. 937–941.
- [10] T. Kourkounakis, "LibriStutter," Dataset, 2021. [Online]. Available: <https://doi.org/10.5683/SP3/NKVOGQ>
- [11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) 2016*, 2016, pp. 779–788.
- [12] J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *International Conference on Machine Learning (ICML) 2021*. PMLR, 2021, pp. 5530–5540.
- [13] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 28 492–28 518.
- [14] J. Harvill, M. Hasegawa-Johnson, and C. D. Yoo, "Frame-level stutter detection," in *Interspeech 2022*, 2022, pp. 2843–2847.
- [15] S. Basu, I. Davidson, and K. Wagstaff, *Constrained clustering: Advances in algorithms, theory, and applications*. Chapman and Hall/CRC, 2008.
- [16] Y. Wang, X. Shen, Y. Yuan, Y. Du, M. Li, S. X. Hu, J. L. Crowley, and D. Vaufreydaz, "TokenCut: Segmenting objects in images and videos with self-supervised transformer and normalized cut," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [17] A. Meyer, S. Ghosh, D. Schindele, M. Schostak, S. Stober, C. Hansen, and M. Rak, "Uncertainty-aware temporal self-learning (UATS): Semi-supervised learning for segmentation of prostate zones and beyond," *Artificial Intelligence in Medicine*, vol. 116, p. 102073, 2021.
- [18] N. B. Ratner and B. MacWhinney, "Fluency Bank: A new resource for fluency research and practice," *Journal of Fluency Disorders*, vol. 56, pp. 69–80, 2018.
- [19] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000.
- [20] Y. Wang, M. Ravanelli, and A. Yacoubi, "Speech emotion diarization: Which emotion appears when?" in *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU) 2023*. IEEE, 2023, pp. 1–7.
- [21] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [22] S. Ghosh, M. Jouaiti, A. Das, Y. Sinha, T. Polzehl, I. Siegert, and S. Stober, "Anonymising elderly and pathological speech: Voice conversion using DDSP and Query-by-Example," in *Interspeech 2024*, 2024, pp. 4438–4442.
- [23] S. P. Bayerl, D. Wagner, I. Baumann, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, "A stutter seldom comes alone – cross-corpus stuttering detection as a multi-label problem," in *Interspeech 2023*, 2023, pp. 1538–1542.
- [24] C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, "Sep-28k: A dataset for stuttering event detection from podcasts with people who stutter," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) 2021*. IEEE, 2021, pp. 6798–6802.
- [25] J. Yamagishi, C. Veaux, K. MacDonald *et al.*, "CSTR VCTK Corpus: English multi-speaker corpus for CSTR voice cloning toolkit (version 0.92)," *University of Edinburgh. The Centre for Speech Technology Research (CSTR)*, 2019.
- [26] M. Bain, J. Huh, T. Han, and A. Zisserman, "Whisperx: Time-accurate speech transcription of long-form audio," in *Interspeech 2023*, 2023, pp. 4489–4493.
- [27] R. Li, J.-Y. Jiang, X. Wu, C.-C. Hsieh, and A. Stolcke, "Speaker identification for household scenarios with self-attention and adversarial training," in *Interspeech 2020*, 2020, pp. 2272–2276.
- [28] S. Cornell, M. S. Wiesner, S. Watanabe, D. Raj, X. Chang, P. Garcia, Y. Masuyam, Z.-Q. Wang, S. Squartini, and S. Khudanpur, "The chime-7 dasr challenge: Distant meeting transcription with multiple devices in diverse scenarios," in *7th International Workshop on Speech Processing in Everyday Environments (CHiME 2023)*, 2023, pp. 1–6.
- [29] D. Wagner, S. P. Bayerl, I. Baumann, E. Noeth, K. Riedhammer, and T. Bocklet, "Large language models for dysfluency detection in stuttered speech," in *Interspeech 2024*, 2024, pp. 5118–5122.
- [30] Y.-J. Shih, Z. Gkalitsiou, A. G. Dimakis, and D. Harwath, "Self-supervised speech models for word-level stuttered speech detection," in *2024 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2024, pp. 937–944.
- [31] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," *Advances in neural information processing systems (NeurIPS) 2020*, vol. 33, pp. 12 449–12 460, 2020.
- [32] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International Conference on Machine Learning (ICML) 2017*. PMLR, 2017, pp. 1321–1330.